

Lesson 8

Logical vs Physical Memory Address

Memory access in operating systems happens through two types of addresses: logical (virtual) and physical. The Memory Management Unit (MMU) sits between the CPU and the physical memory. Its job is to translate every logical address into the correct physical address.

- Programs to believe they have a large, continuous memory space
- The OS protects one process's memory from another process
- Efficient use of RAM through paging, segmentation, or both

Whenever the CPU accesses memory, it sends a logical address → the MMU converts it → the corresponding physical address in RAM is accessed.

Logical Address

A logical address is generated by the CPU while a program runs. It represents the address from the process's perspective and does not exist physically, hence it is also called a virtual address.

- The logical address space is the set of all logical addresses a process can generate.
- Programs use logical addresses to reference memory, and the MMU translates them into physical addresses when accessing actual memory.

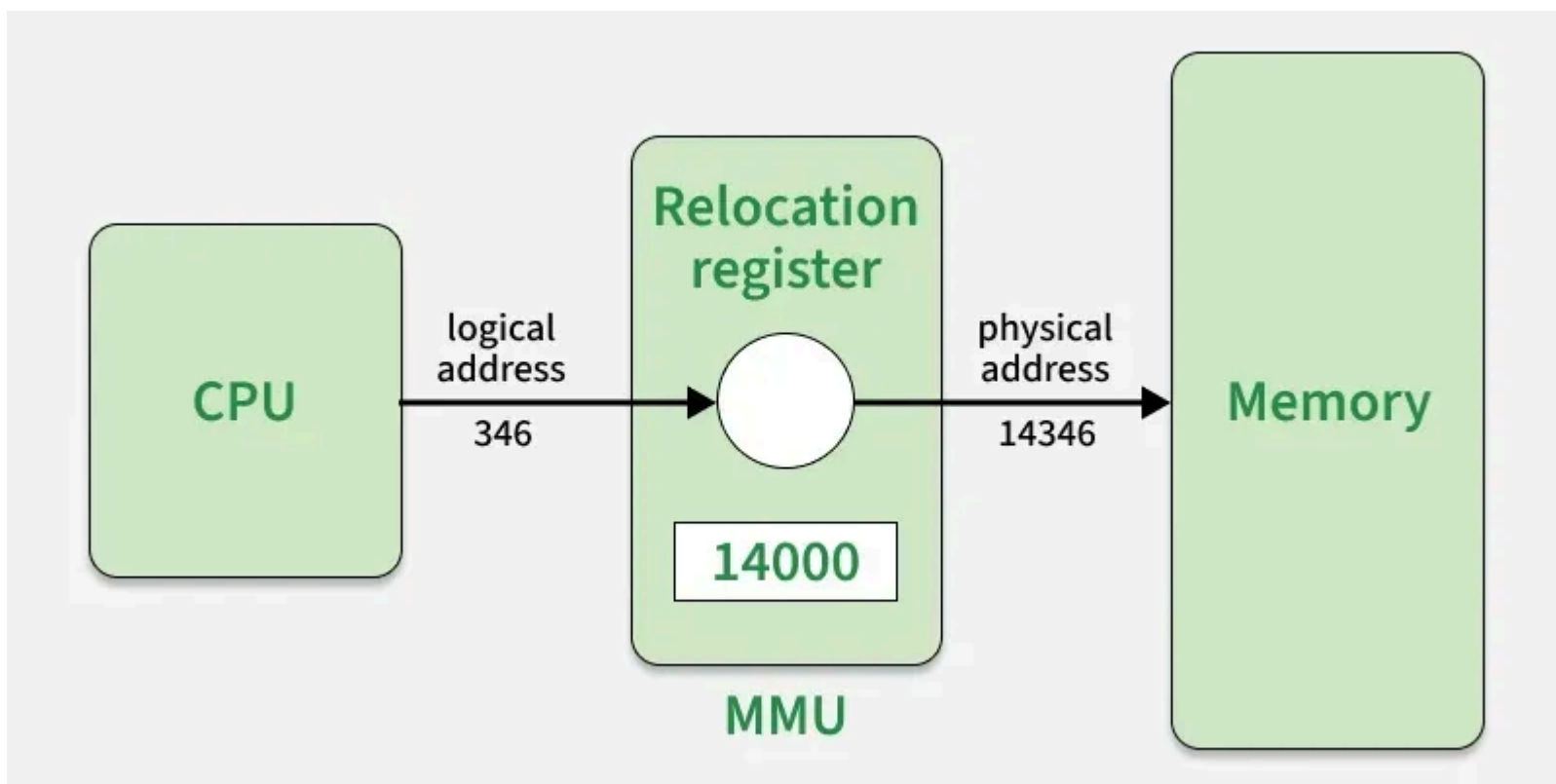
Physical Address

A physical address is the real location in main memory (RAM) where data or instructions are stored. The physical address space consists of all physical addresses corresponding to logical addresses.

- The MMU performs address translation using a page table, mapping each logical page to a physical frame.
- This allows processes to access memory transparently, without knowing actual memory locations.

Similarities in Logical and Physical Addresses

- Both logical and physical addresses are used to identify a specific location in memory.
- Each address type can be represented in different formats such as binary, hexadecimal, or decimal.
- They have a finite range, which is determined by the number of bits used to represent them.



Memory Management

Memory Management Unit

The Memory Management Unit (MMU) is a hardware component in the computer system that handles all memory and caching operations associated with the CPU. Its main function is to translate logical (virtual) addresses generated by the CPU into physical addresses in main memory.

Key functions of Memory Management Unit

- Logical addresses provide abstraction, so processes don't need to know physical locations.
- These logical addresses are translated into physical addresses using a page table.
- Translation is transparent and handled by hardware (MMU).
- Enables efficient memory management through paging and segmentation.

Logical Address vs. Physical Address

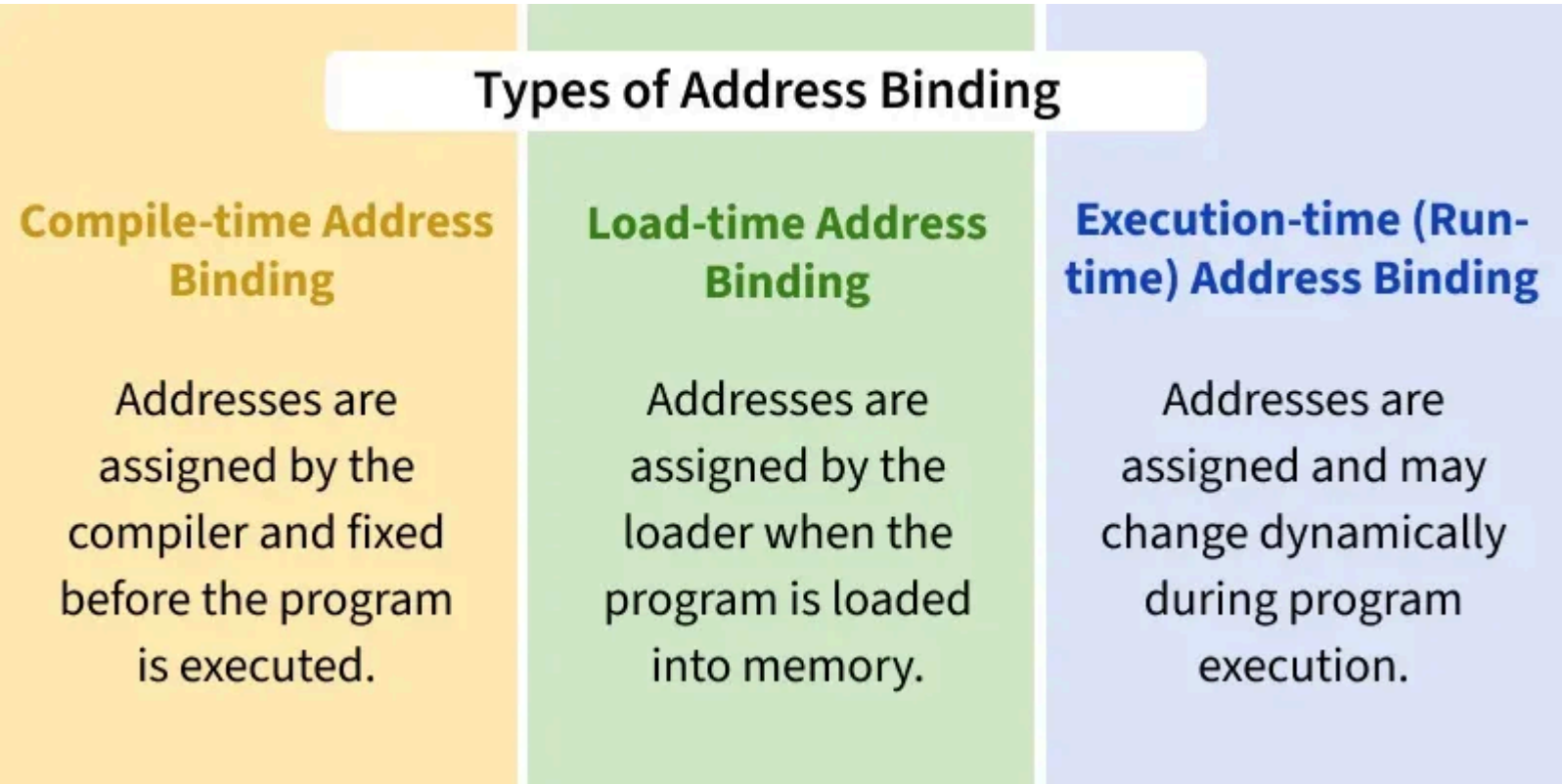
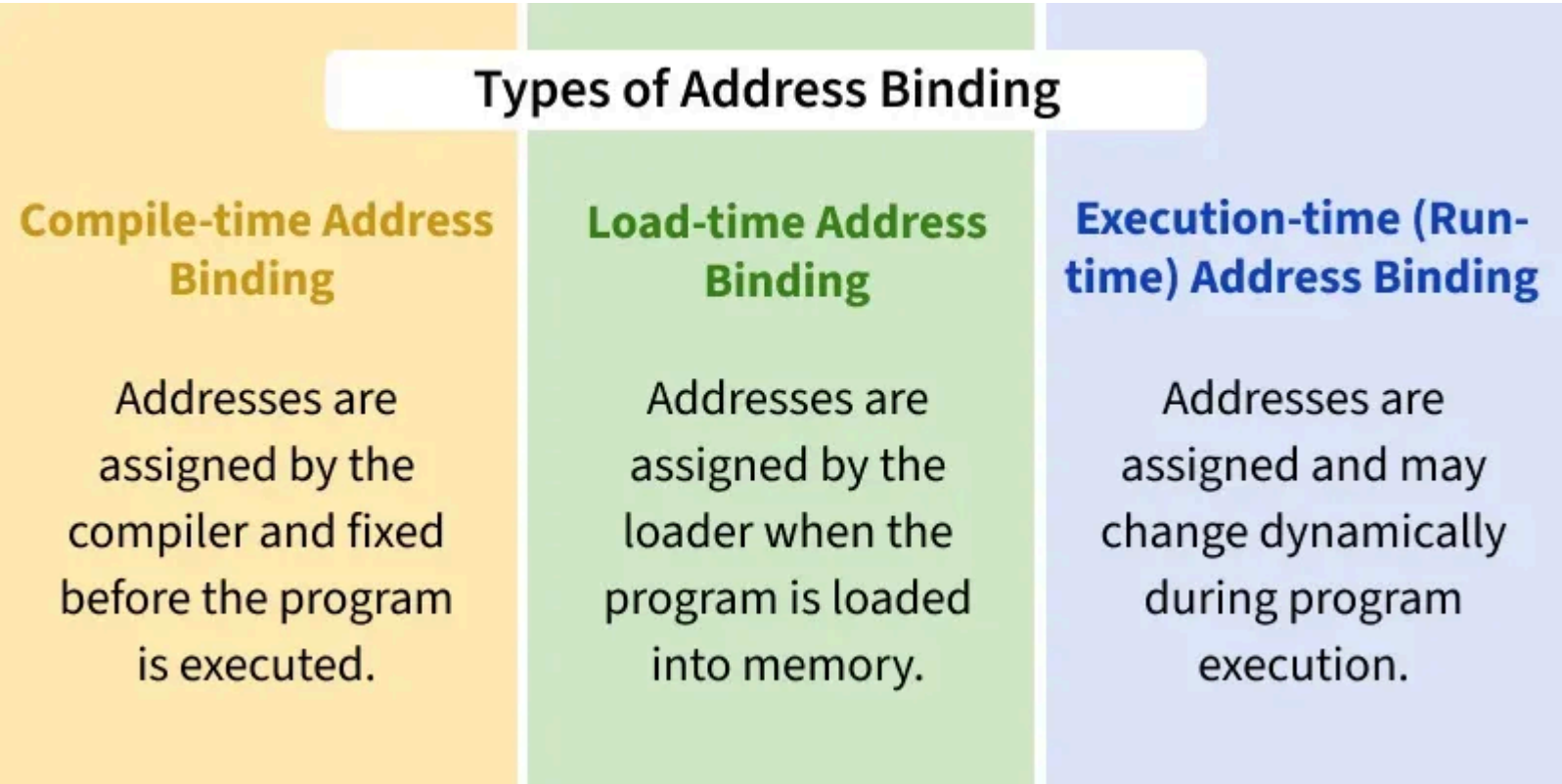
Logical Address	Physical Address
Generated by the CPU during program execution	Generated by the Memory Management Unit (MMU)
Logical Address Space is set of all logical addresses generated by CPU in reference to a program.	Physical Address is set of all physical addresses mapped to the corresponding logical addresses.
User can view and access the logical address of a program.	User can never view physical address of program.
Can change during program execution (due to relocation, paging, etc.)	Generally fixed once assigned in memory
The user can use the logical address to access the physical address.	The user can indirectly access physical address but not directly.
Logical address can be change.	Physical address will not change.

Logical Address	Physical Address
Virtual address.	Real address.

Address Binding

Address Binding is the mapping of a logical (virtual) address to a physical address. It allocates a physical memory region to a logical pointer. It is the process of assigning physical memory addresses to program instructions and data. The OS handles this aspect of computer memory management on behalf of programs that need memory access.

- Executes at compile time, load-time, or execution-time.
- The linker and loader assign actual memory addresses.
- Allows code relocation in memory.
- Supports dynamic memory allocation at runtime.



Example: Consider a program that generates the following logical addresses during execution:

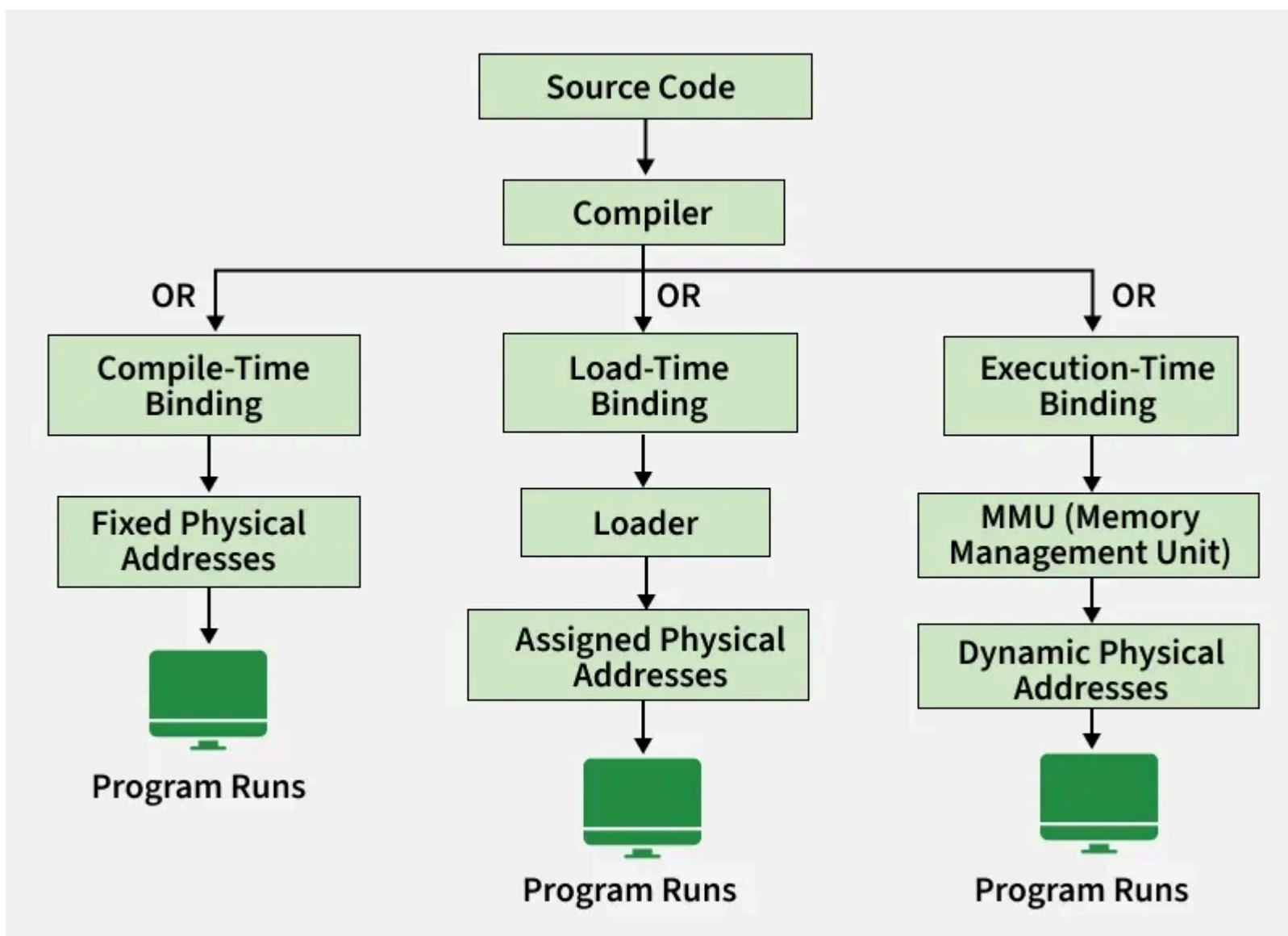
- Logical Addresses: 0, 4, 8
- Base (physical) address: 1000

Logical Address	Physical Address
0	1000
4	1004
8	1008

Address Binding Rule: Physical Address = Base Address + Logical Address

Types of Address Binding

The mapping of data and computer instructions to actual memory locations is known as address binding. In computer memory, logical and physical addresses are employed.



Divided into three types, but only one method can be applied.

Compile-Time Address Binding

Compile-time address binding assigns fixed physical addresses to symbolic addresses during compilation. Functions and global variables occupy the same memory locations at runtime. This method is simple and efficient but cannot adapt to runtime changes.

- Done by the compiler during compilation.
- Symbolic addresses linked to fixed physical addresses.
- Addresses determined before program execution.
- Suitable for functions and global variables.
- Simple and efficient, but not adjustable at runtime.

Load Time Address Binding

Load Time Address Binding occurs when a program is loaded into memory. The loader assigns physical addresses, and the linker resolves external references. More flexible than compile-time binding, allowing memory adjustments and support for dynamic libraries.

- Take place when the program is loaded into memory.
- Performed by the OS memory manager (loader).
- Loader assigns physical memory addresses during loading.
- More flexible than compile-time binding.
- Allows memory adjustments at load time.
- Supports dynamic libraries

Execution Time or Dynamic Address Binding

Execution Time Address Binding occurs during program execution, allowing memory locations to change. Supports dynamic memory allocation, late binding, and polymorphism. Used in dynamic and object-oriented languages, with modern OSs like Windows, Linux, and Unix.

- Performs during program execution.
- Program memory locations may change at runtime.
- Memory can be allocated and deallocated dynamically.
- Most flexible type of binding.
- Common in dynamic and object-oriented languages.

Dynamic loading and linking

An operating system acts as an intermediary between the user of a computer and computer hardware. The purpose of an operating system is to provide an environment in which a user can execute programs conveniently and efficiently.

Linking and Loading are utility programs that play an important role in the execution of a program. Linking intakes the object codes generated by the assembler and combines them to generate the executable module. On the other hand, the loading loads this executable module to the main memory for execution.

What is Loading?

To bring the program from secondary memory to main memory is called Loading. It is performed by a loader. It is a special program that takes the input of executable files from the **linker**, loads it to the main memory, and prepares this code for execution by a computer. There are two types of loading in the operating system:

- **Static Loading:**
 - Loading the entire program into the main memory before the start of the program execution is called static loading.

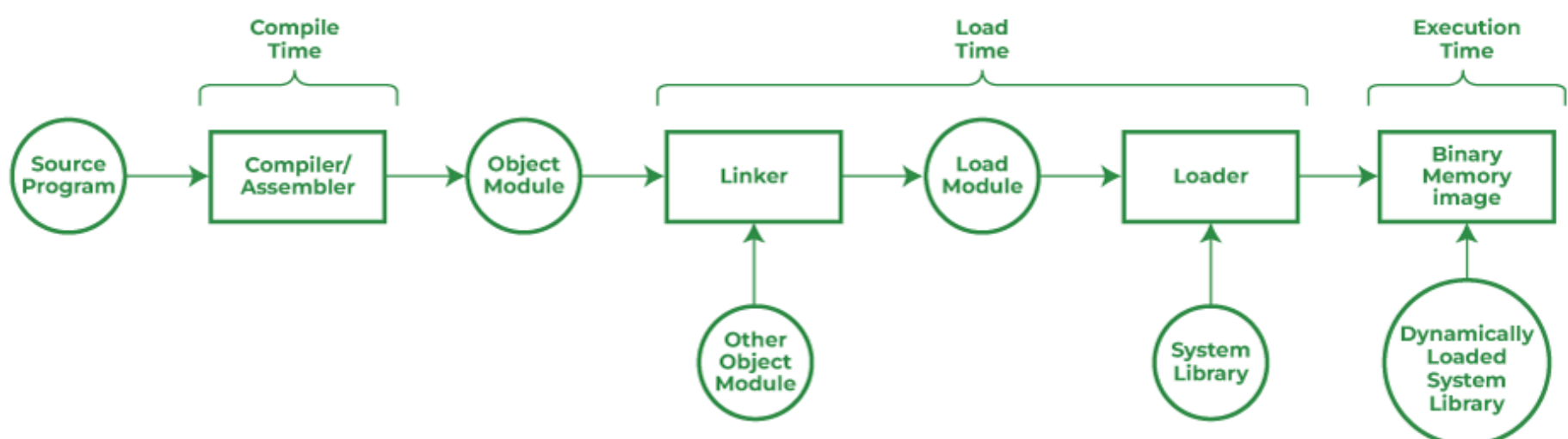
- If static loading is used then accordingly static linking is applied.
- **Dynamic Loading:**
 - Loading the program into the main memory on demand is called dynamic loading.
 - If dynamic loading is used then accordingly dynamic linking is applied.

What is Linking?

Establishing the linking between all the modules or all the functions of the program in order to continue the program execution is called linking. Linking is a process of collecting and maintaining pieces of code and data into a single file. Linker also links a particular module into the system library. It takes object modules from the assembler as input and forms an executable file as output for the loader. Linking is performed at both compile time when the source code is translated into machine code, and load time, when the program is loaded into memory by the loader. Linking is performed at the last step in compiling a program.

- **Static Linking:**
 - A statically linked program takes constant load time every time it is loaded into the memory for execution.
 - Static linking is performed by programs called linkers as the last step in compiling a program. Linkers are also called link editors.
 - In static linking, if any of the external programs has changed then they have to be recompiled and re-linked again else the changes won't reflect in the existing executable file.
- **Dynamic Linking:**
 - Dynamic linking is performed at run time by the operating system.
 - In dynamic linking, this is not the case and individual shared modules can be updated and recompiled. This is one of the greatest advantages dynamic linking offers.
 - In dynamic linking load time might be reduced if the shared library code is already present in memory.

For more information please refer to the [Static and Dynamic Linking in Operating Systems](#) article.



Differences between Linking and Loading

Linking	Loading
The process of collecting and maintaining pieces of code and data into a single file is known as Linking in the operating system.	Loading is the process of loading the program from secondary memory to the main memory for execution.
Linking is used to join all the modules.	Loading is used to allocate the address to all executable files and this task is done by the loader.
Linking is performed with the help of Linker. In an operating system , Linker is a program that helps to link object modules of a program into a single object file. It is also called a link editor.	A loader is a program that places programs into memory and prepares them for execution.
Linkers are an important part of the software development process because they enable separate compilation. Apart from that organizing a large application as one monolithic source file, we can decompose it into smaller, more manageable modules that can be modified and compiled separately.	The loader is responsible for the allocation, linking, relocation, and loading of the operating system.

Contiguous and non-contiguous memory management

Feature	Contiguous memory allocation	Non-contiguous memory allocation
Storage Style	The process is stored in one single block of RAM	The process is divided into parts and scattered in RAM
Hardware	Requires simple Base and limit registers	Require a Page table or Segment table and specialized hardware(MMU)
Fragmentation	Sufferes heavily from external fragmentation	Eliminates External Fragmentation; may have tiny Internal fragmentation
OS overhead	Low. The OS only tracks the start address and size	High. The OS must maitain a mapping table for every single process
Execution	Faster address translation(Simple addition)	Slightly slower due to page lookup(minimized by TLB hardware)
Swapping	Difficult; the OS must find a single large hole to bring a process back	Easy, the OS can bring pieces into any available small frames.

Placement Algorithms

In an operating system (OS), memory management is a critical function that ensures efficient allocation and utilization of memory resources. When processes are initiated, they require

execution time and storage space, which is allocated in memory blocks. The OS uses various memory allocation algorithms to manage the assignment of memory to these incoming processes.

The following are the most used algorithms:

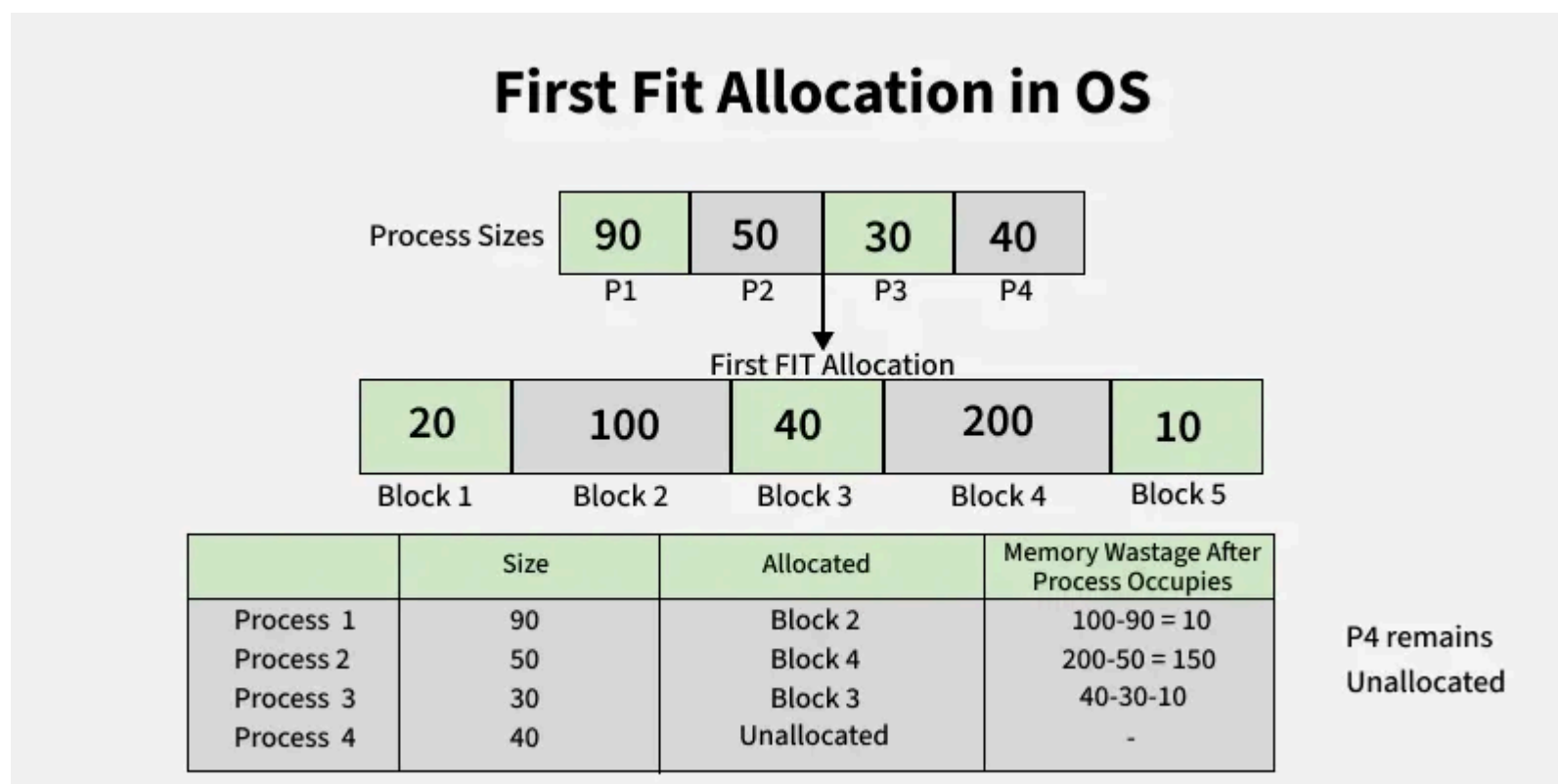
1. **First Fit**
2. **Best Fit***
3. **Worst Fit***
4. **Next Fit***

1. First Fit

The **First Fit** method works by searching through memory blocks from the beginning, looking for the first block that can fit the process.

- For any process P_n , the operating system looks through the available memory blocks.
- It allocates the first block that is free and large enough to hold the process.

In simple terms, First Fit simply finds the first available block that can fit the process and assigns it there. It's a straightforward and fast way to allocate memory.



Algorithm for First-Fit Memory Allocation Scheme

Steps:

1. Start with the first process in the list.
2. For each process P_n , do the following:
 - **Step 1:** Search through the available memory blocks, starting from the first block.
 - **Step 2:** Check if the current memory block is free and has enough space to accommodate P_n .

- **Step 3:** If a suitable block is found, allocate it to the process and update the block's status (mark it as allocated).
- **Step 4:** If no suitable block is found, mark the process as unallocated or waiting.

3. Repeat the above steps for each process.

Pseudocode:

```
FirstFit(memory_blocks[], processes[]):
for each process Pn in processes:
for each memory block B in memory_blocks:
if B is free and B.size >= Pn.size:
Allocate B to Pn
Mark B as occupied
Break out of the loop and move to next process
if no block found:
Mark Pn as unallocated
```

Advantages of First Fit Algorithm

The First Fit algorithm in operating systems offers several benefits:

1. It is straightforward to implement and easy to understand, making it ideal for systems with limited computational power.
2. Memory can be allocated quickly when a suitable free block is found at the start.
3. When processes have similar memory sizes, First Fit can help minimize fragmentation by utilizing the first available block that fits the process.

Disadvantages of First Fit Algorithm

Despite its advantages, the First Fit algorithm has a few downsides:

1. Over time, First Fit can lead to both external fragmentation, where small free memory blocks are scattered, and internal fragmentation, where allocated memory exceeds the process's requirement, wasting space.
2. It may not always allocate memory in the most efficient manner, leading to suboptimal use of available memory.
3. For large processes, First Fit can be less efficient, as it may need to search through numerous smaller blocks before finding an appropriate one, which can slow down memory allocation.

2. Best Fit

Best-Fit Allocation is a memory allocation technique used in operating systems to allocate memory to a process. In Best-Fit, the operating system searches through the list of free blocks

of memory to find the block that is closest in size to the memory request from the process. Once a suitable block is found, the operating system splits the block into two parts: the portion that will be allocated to the process, and the remaining free block.

Advantages of Best-Fit Allocation include improved memory utilization, as it allocates the smallest block of memory that is sufficient to accommodate the memory request from the process. Additionally, Best-Fit can also help to reduce memory fragmentation, as it tends to allocate smaller blocks of memory that are less likely to become fragmented.

Disadvantages of Best-Fit Allocation include increased computational overhead, as the search for the best-fit block of memory can be time-consuming and requires a more complex search algorithm. Additionally, Best-Fit may also result in increased fragmentation, as it may leave smaller blocks of memory scattered throughout the memory space.

Overall, Best-Fit Allocation is a widely used memory allocation technique in operating systems, but its effectiveness may vary depending on the specifics of the system and the workload being executed.

For both [fixed and dynamic memory allocation schemes](#), the operating system must keep list of each memory location noting which are free and which are busy. Then as new jobs come into the system, the free partitions must be allocated.

This method keeps the free/busy list in order by size - smallest to largest. In this method, the operating system first searches the whole of the memory according to the size of the given job and allocates it to the closest-fitting free partition in the memory, making it able to use memory efficiently. Here the jobs are in the order from smallest job to largest job.

Job Number	Memory Requested
J1	20 K
J2	200 K
J3	500 K
J4	50 K

Memory location	Memory block size	Job number	Job size	Status	Internal fragmentation
10567	30 K	J1	20 K	Busy	10 K
30457	50 K	J4	50 K	Busy	None
300875	200 K	J2	200 K	Busy	None
809567	700 K	J3	500 K	Busy	200 K
Total available :	980 K	Total used :	770 K		210 K

As illustrated in above figure, the operating system first search throughout the memory and allocates the job to the minimum possible memory partition, making the memory allocation efficient.

Advantages of Best-Fit Allocation :

- Memory Efficient. The operating system allocates the job minimum possible space in the memory, making memory management very efficient.
- To save memory from getting wasted, it is the best method.
- Improved memory utilization
- Reduced memory fragmentation
- Minimizes external fragmentation

Disadvantages of Best-Fit Allocation :

- It is a Slow Process. Checking the whole memory for each job makes the working of the operating system very slow. It takes a lot of time to complete the work.
- Increased computational overhead
- May lead to increased internal fragmentation
- Can result in slow memory allocation times.

Best-fit allocation is a memory allocation algorithm used in operating systems to allocate memory to processes. In this algorithm, the operating system searches for the smallest free block of memory that is big enough to accommodate the process being allocated memory.

Here is a brief overview of the best-fit allocation algorithm:

1. The operating system maintains a list of all free memory blocks available in the system.
2. When a process requests memory, the operating system searches the list for the smallest free block of memory that is large enough to accommodate the process.
3. If a suitable block is found, the process is allocated memory from that block.
4. If no suitable block is found, the operating system can either wait until a suitable block becomes available or request additional memory from the system.
5. The best-fit allocation algorithm has the advantage of minimizing external fragmentation, as it searches for the smallest free block of memory that can accommodate a process. However, it can also lead to more internal fragmentation, as processes may not use the entire memory block allocated to them.

3. Worst Fit

In **Worst-Fit Memory Allocation** allocation technique, the process traverses the whole memory and always search for the largest hole/partition, and then the process is placed in that hole/partition. It is a slow process because it has to traverse the entire memory to search the largest hole.

Algorithm for Worst Fit Memory Management Scheme

Step 1: Input the list of memory blocks along with their sizes.

Step 2: Input the list of processes with their sizes.

Step 3: For each process, check for the largest memory block (i.e., the block with the maximum size) that is large enough to accommodate the process.

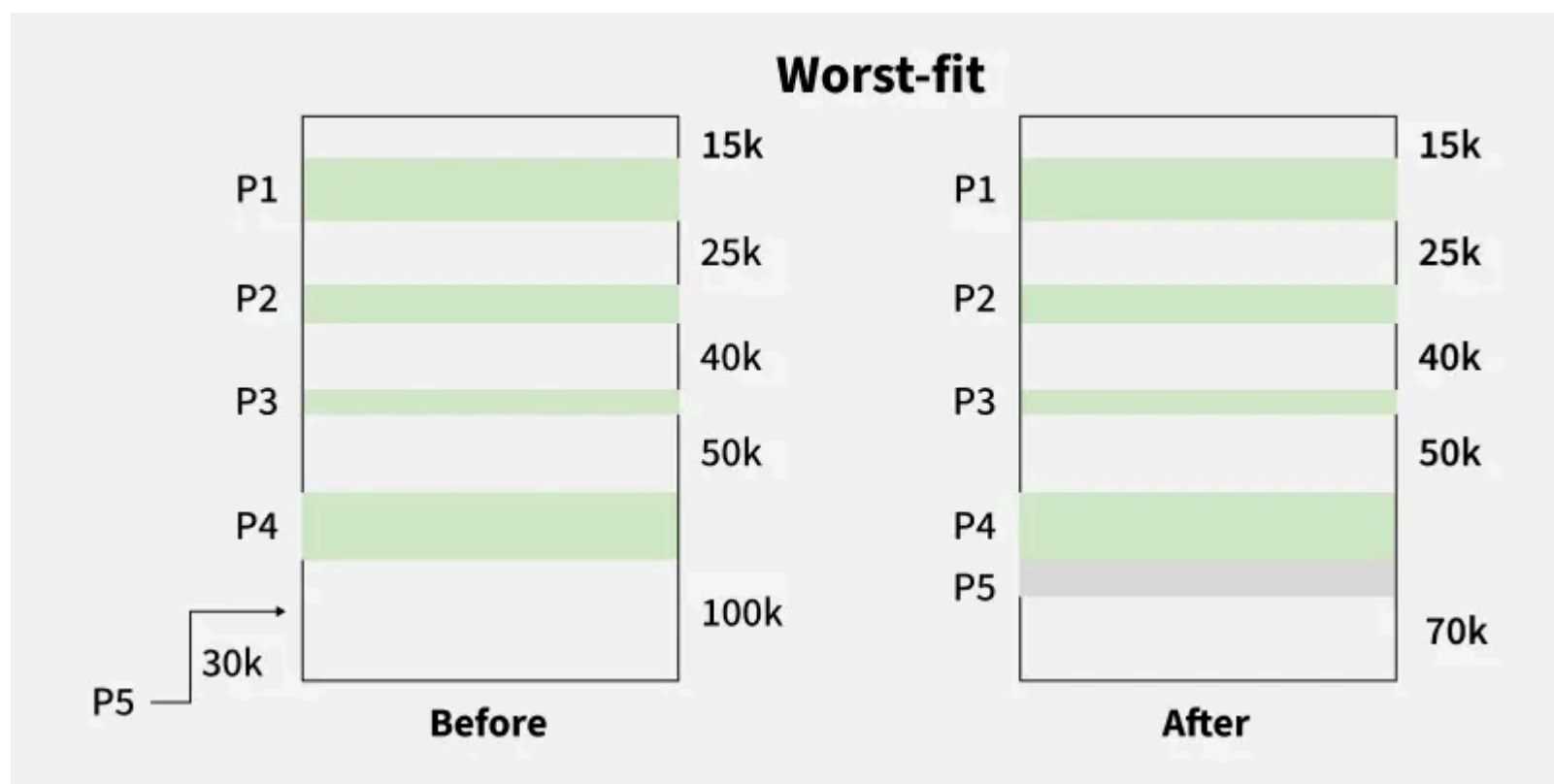
Step 4: If such a block is found, allocate the process to that block and update the block size.

Step 5: If no suitable block is found, leave the process unallocated and move on to the next process.

Step 6: Repeat the above steps for all processes.

Step 7: Stop.

[Refer Program for Worst Fit algorithm in Memory Management for implementation](#)



worst-fit

Advantages of Worst-Fit Allocation

- **Reduces Chances of Small Fragments:** By allocating the largest available block, worst fit leaves larger leftover fragments, which are more likely to be useful for future allocations.
- **Simple to Implement:** The logic is straightforward: find the largest block that can fit the process and allocate it.
- **Efficient for Large Processes:** It always looks for the biggest space, large processes may find better fits compared to other strategies like Best Fit or First Fit.

Disadvantages of Worst-Fit Allocation

- **External Fragmentation:** **Worst-Fit Allocation** can lead to significant ****external fragmentation****. By allocating the largest available block, it often leaves behind many smaller, unusable fragments of memory.
- **Inefficient Memory Usage:** Allocating the largest block to a process, even if it's much larger than required, can lead to **inefficient memory usage**.
- **Lower Allocation Time:** **Worst-Fit** requires the operating system to search for the largest available block, which can be slower than methods like **First-Fit** or **Next-Fit**, especially if there are many free blocks.

External Fragmentation

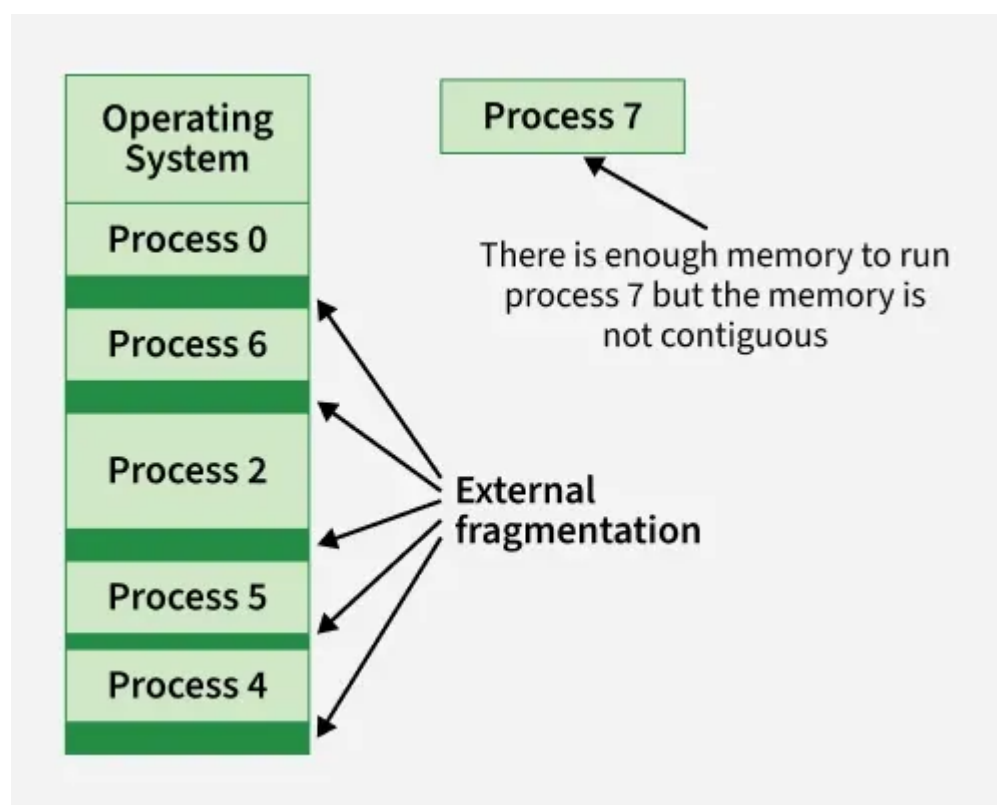
External fragmentation is a problem in memory management where free memory is divided into small, non-contiguous blocks. Even though there may be enough total free memory to run a new program, the memory is scattered in tiny pieces, so it's impossible to find a single, large block for the program to use. This leads to wasted memory and poor system performance.

Cause of External Fragmentation

External fragmentation is caused by the dynamic allocation and deallocation of processes. Imagine memory as a long parking lot.

1. A few cars (processes) park in different spots.
2. Then, some of these cars leave, creating empty spaces (free memory).
3. These empty spaces are of different sizes and are scattered throughout the lot.
4. If a new, large truck (a new process) arrives, it can't find a single, continuous space big enough to park, even though the total empty space in the lot is more than enough.

This is exactly how external fragmentation works. The "holes" in the memory are too small and separated to be useful for larger processes.



How to Avoid External Fragmentation?

To avoid external fragmentation in an operating system:

- **Paging/Segmentation:** Paging allocates memory in fixed-size pages, avoiding external fragmentation. Segmentation uses variable-sized blocks but may still leave some gaps.
- **Compaction:*** Rearranges processes in memory to merge free spaces into larger contiguous blocks.
- **Best-Fit Allocation:** Assigns the smallest available block that fits a process, reducing wasted space (though not eliminating fragmentation completely).

Illustrative Example of External Fragmentation

Lets consider a memory space having 4 processes, each of them requires different amount of memory to execute, as shown in Figure 1.

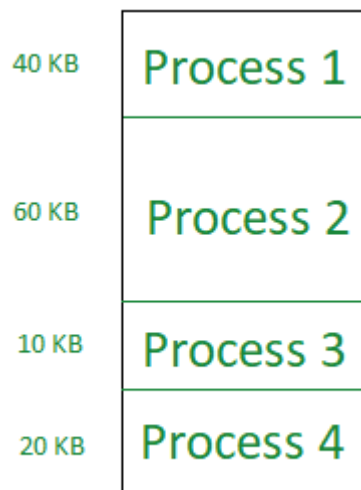


Figure 1

Now suppose Process 1 and Process 3 have been completed, so now there are free places available, as shown in Figure 2.

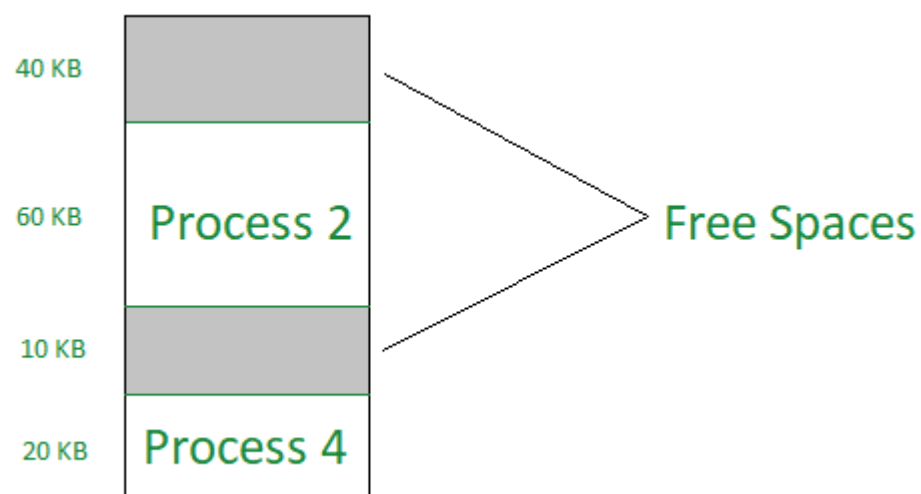


Figure 2

Now, if we want to run another process (Process 5) requiring memory 50 KB, we will not be able to do it, although there is enough memory to run Process 5 as the memory is not contiguous, as shown in Figure 3.

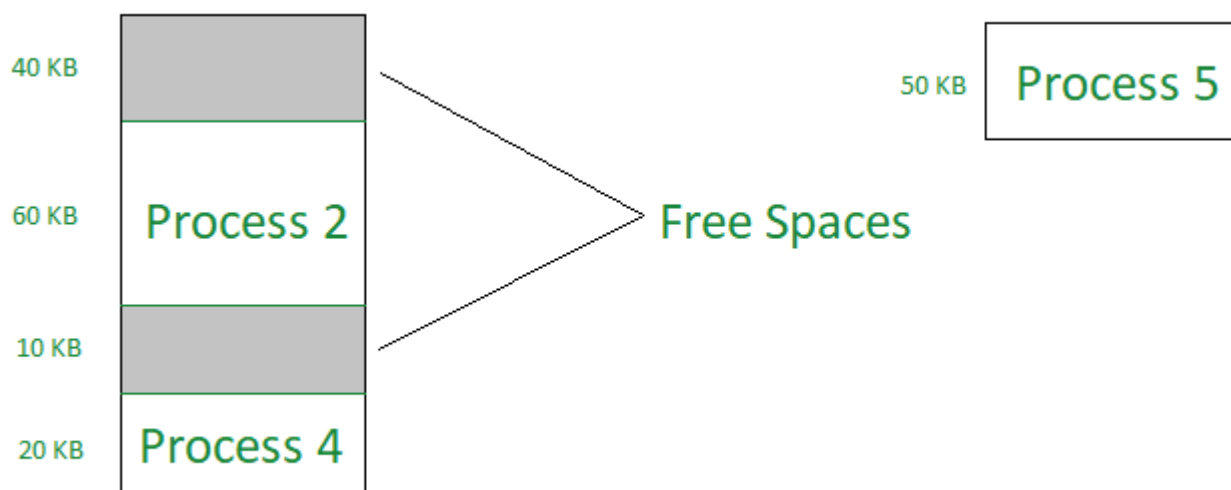


Figure 3

That's how external fragmentation occurs.

Causes of External Fragmentation

The primary causes of external fragmentation include:

- **Variable Process Sizes:** Different processes need different amounts of memory, leaving uneven gaps when loaded/unloaded.
- **Allocation & Deallocation:** When processes finish, freed blocks may not be adjacent, creating scattered holes.
- **Non-Uniform Release:** Memory released at irregular points leads to gaps of unpredictable sizes.
- **Allocation Strategies:** Best-fit and worst-fit may worsen fragmentation by leaving behind many small unusable holes.
- **Fixed Partitioning:** Fixed-sized partitions cause mismatches between process size and partition size, leading to internal and external fragmentation.
- **High Process Turnover:** Frequent loading and removal of processes leaves memory fragmented over time.

Solution of External Fragmentation

Below mentioned are the solutions of External Fragmentation that are mentioned below.

1. Memory Compaction

Memory compaction includes rearranging the items in memory to unite free memory hinders and make bigger contiguous memory regions. This is the closely guarded secret:

- **Process Relocation:** Moves processes closer together to eliminate or reduce gaps, creating larger contiguous free memory blocks.
- **Benefits:** Directly reduces external fragmentation, increasing the chance of allocating memory without gaps.
- **Challenges:** Can be resource-intensive, may disrupt running processes, and frequent compaction can impact system performance.

2. Paging

Paging is a memory management technique that divides physical memory into fixed-size blocks called pages. It helps overcome external fragmentation and simplifies memory allocation.

- **Page Size:** Memory is divided into equal-sized pages, preventing small unusable gaps.
- **Virtual Memory:** Allows processes to use non-contiguous physical memory while seeing it as a single contiguous space.
- **Page Table:** The OS maintains a page table that maps virtual addresses to physical addresses.

- **Page Replacement:** When memory is full, pages can be swapped between main memory and disk to free space.

Advantages of External Fragmentation

- **Flexibility:** Variable-sized allocation lets processes request memory as needed, not limited to fixed blocks.
- **Efficient Utilization:** Small gaps may appear, but overall memory use can still be better than fixed-sized allocation.
- **Simplicity:** Strategies like first-fit or best-fit are easy and fast to implement, reducing overhead.
- **Dynamic Growth:** Supports applications that expand memory usage over time by allocating differently sized blocks.

Disadvantages of External Fragmentation

- **Memory Consumption:** Small, unusable intervals of memory accumulate, making it difficult to allocate large chunks of memory even if there is sufficient free space
- **Reduced Performance:** Over time, fragmentation can slow down memory allocation processes, as the system must search for enough free space.
- **Need for Constant Wear:** To prevent cracking, the system control may need to rotate (compress) repeatedly, which can take time to wear down
- **Limits Program Growth :** As memory fragmentation occurs, it can prevent the memory usage of programs from expanding, causing inefficiency or corruption

Summary

1. Logical vs. Physical Memory Address

- **Logical (Virtual) Address:** Generated by the CPU during program execution. It represents memory from the program's perspective and can change over time. Users can view and access it.
- **Physical Address:** The actual, real location in the physical RAM. It is generated and handled by the hardware, and it generally remains fixed once assigned. Users cannot view it directly.
- **Memory Management Unit (MMU):** A hardware component that sits between the CPU and RAM. It transparently translates logical addresses into physical addresses (often using a page table).

2. Address Binding

Address binding is the process of mapping program instructions and data (logical pointers) to actual physical memory locations. It can happen at three different stages:

- **Compile-Time:** The compiler assigns fixed physical addresses during compilation. It is simple but completely inflexible at runtime.
- **Load-Time:** The OS loader assigns physical addresses when the program is loaded into memory. This provides better flexibility and supports dynamic libraries.
- **Execution-Time (Dynamic):** Binding happens while the program is actively running. Memory locations can change dynamically, making this the most flexible method (used by modern OSs like Linux and Windows).

3. Dynamic Loading and Linking

- **Loading (Moving code from secondary memory to RAM):**
 - *Static:* Loads the entire program into main memory before execution starts.
 - *Dynamic:* Loads program modules into main memory on-demand at runtime.
- **Linking (Combining code modules into a single execution file):**
 - *Static:* Combines all external modules at compile time. If an external module changes, the whole program must be recompiled.
 - *Dynamic:* Performed at runtime by the OS. Shared modules can be updated independently without breaking the executable.

4. Contiguous vs. Non-Contiguous Memory

- **Contiguous Allocation:** Stores a process in a single, unbroken block of RAM. It features faster address translation but suffers heavily from external fragmentation and makes swapping difficult.
- **Non-Contiguous Allocation:** Breaks a process into parts and scatters them across RAM. It requires an MMU and page tables (higher OS overhead), but it eliminates external fragmentation and simplifies swapping.

5. Memory Placement Algorithms

When a process needs memory, the OS uses one of these algorithms to find an available block:

Algorithm	How it Works	Pros	Cons
First Fit	Searches from the beginning and allocates the <i>first</i> block that is big enough.	Fast and easy to implement.	Causes fragmentation over time.
Best Fit	Searches the entire memory to find the <i>smallest</i> block that fits the process.	Memory efficient; minimizes wasted space.	Slow process; can leave behind tiny, unusable holes.
Worst Fit	Searches the entire memory and places the process in the <i>largest</i> available block.	Leftover fragments are large enough to be useful later.	Slow search times; can cause severe external fragmentation.

6. External Fragmentation

- **The Problem:** A state where total free memory is sufficient to host a process, but the memory is split into tiny, non-contiguous blocks scattered across RAM, making it impossible to allocate a single contiguous block.
- **Primary Causes:** Dynamic loading/unloading of processes, variable process sizes, and certain placement strategies (like Best-Fit/Worst-Fit) that leave irregular gaps.
- **Solutions:**
 - **Memory Compaction:** Shifting running processes closer together in memory to merge scattered "holes" into one large block (resource-heavy).
 - **Paging:** Dividing physical memory into fixed-size blocks (pages), which inherently eliminates external fragmentation by allowing non-contiguous allocation.